Coordination-Free Schema Evolution for Replicated Property Graphs

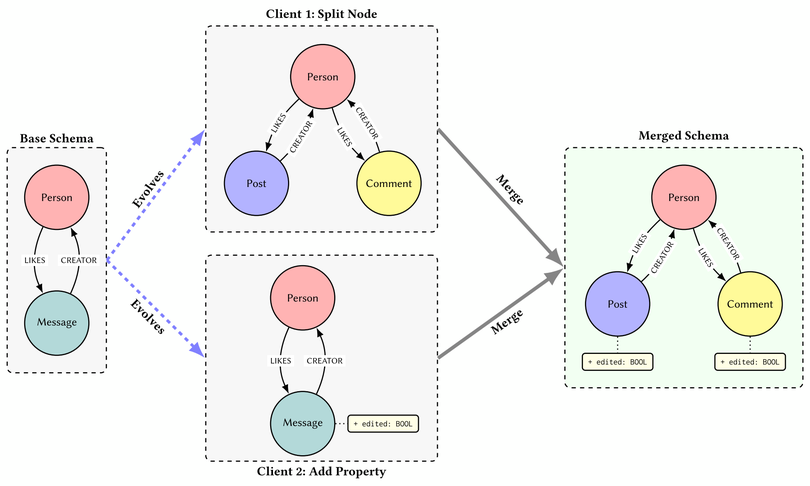
Michael Landgraf[1] Annette Bieniusa[1] Stefania Dumbrava[2],
landgram@rptu.de bieniusa@rptu.de stefania.dumbrava@ensiie.fr,
[1] RPTU Kaiserslautern-Landau; [2] ENSIIE, Inria Paris, IRIF, SAMOVAR (Télécom SudParis)

REFRACT Middleware

1 Concurrent Schema Evolution

Independent edits, shared meaning.

- Evolve schemas locally
- No central coordinator
- Converge to one view
- No data rewrite



Same updates ⇒ same schema.

Example: Client 1 splits `Message`, Client 2 adds `edited`; after merging, both `Post` and `Comment` carry `edited`.

2 Schema Modification Operators

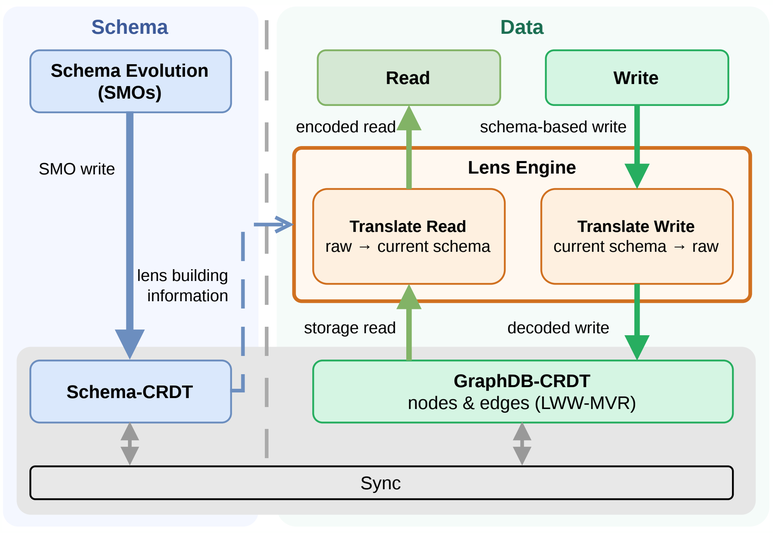
REFRACT adapts the newly released Neo4j `GRAPH TYPE` [6] schema language and lifts the SMOs of Bonifati et al. [1] onto it, which include: CREATE, DROP, RENAME, CHANGE, JOIN and SPLIT

3 Schema as a Replicated Convergent Object

Schema-CRDT
Stores and merges the replicated property-graph schema.

GraphDB-CRDT
Stores and merges the replicated property-graph data.

Lens Engine
Translates reads and writes under the current schema.



REFRACT Architecture: separating schema management from data management.

4 Example Schema and Data Conflict

	SHARED START	CONCURRENT: SCHEMA EDIT →(happens-before)→ DATA WRITE		AFTER SYNCHRONISATION
	Initial state →	**Replica C1**	**Replica C2**	→ **Both replicas**
Schema types + keys	`lastName: Str` `age: Num`	`name: Str` `age: Str`	`famName: Str` `age: Bool`	`famName: Str` `age: Str`
Data raw keys + values	`lastName: 'Hacker'` `age: 24`	`lastName: 'Anon'` `age: '25'`	`lastName: 'Coder'` `age: FALSE`	`lastName: 'Coder'` `age: '25'`
View lens output	`lastName: 'Hacker'` `age: 24`	`name: 'Anon'` `age: '25'`	`famName: 'Coder'` `age: FALSE`	`famName: 'Coder'` `age: '25'`

Same delivered updates → same schema and projected view.
Merged data shows resolved values; concurrent writes remain stored.

Concurrent `RENAME` is ordered by LWW, concurrent retyping by the **type lattice**. Both replicas end at the same projected view.

Type Lattice Concurrent retypings of the same property merge to the upper bound in the total order of types:

`Bool` ≺ `Date` ≺ `Number` ≺ `String`

5 Convergence

Schema and graph data are replicated using Yjs CRDTs [4]. Indexes, lenses and read operations are deterministic functions of this state, so REFRACT inherits strong eventual consistency (SEC) [7].

Experimental Evaluation

Experimental Setup

Benchmark: LDBC Social Network Benchmark [3] (SF3 samples of `Person` and `KNOWS`); 8 of 19 SMOs evaluated; YCSB [2] for replication workloads.
Setup: Ryzen 7 5800X, 32 GB RAM; Node.js and Yjs, single machine.
Baselines: Eager migration on GraphDB-CRDT and Cambria [5].

Aspect	Eager migration GraphDB-CRDT	Cambria [5]	REFRACT
Data model	Property graphs	JSON documents	**Property graphs**
Migration	Rewrites affected data	Bidirectional lenses	**Bidirectional lenses**
Schema merge	not supported	translation only	**Schema-CRDT**

Highlights

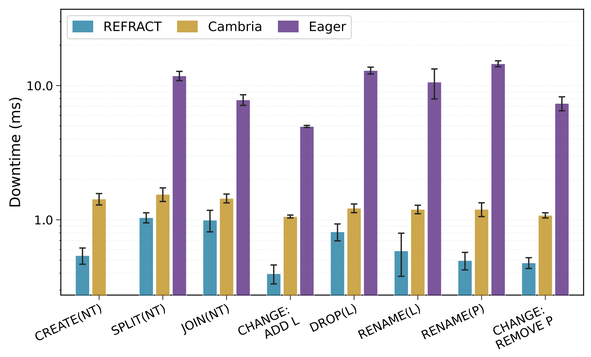
90–492×
lower schema downtime at 10K nodes against eager migration

0.03 %
schema storage overhead at 10K nodes (constant 11 KB, fixed schema)

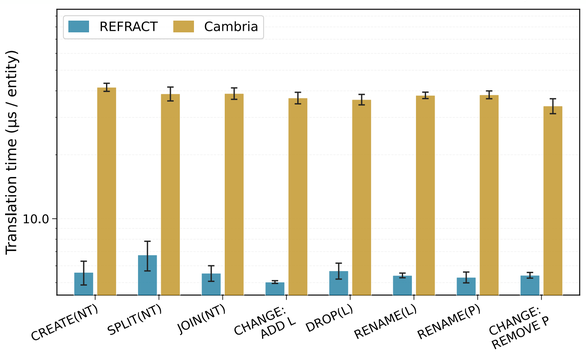
Steady
throughput while a schema evolution runs under a live read-heavy workload

Cost of a Schema Evolution (log-scale)

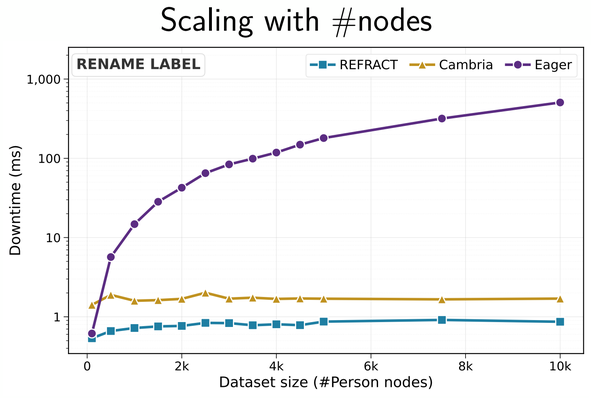
Downtime per SMO (1K nodes)



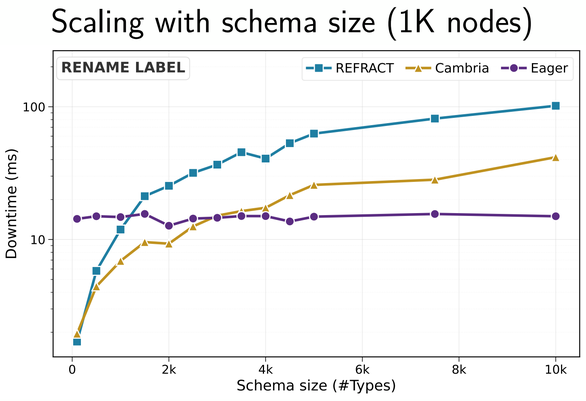
Per-node lens translation (1K nodes)



Scaling with #nodes

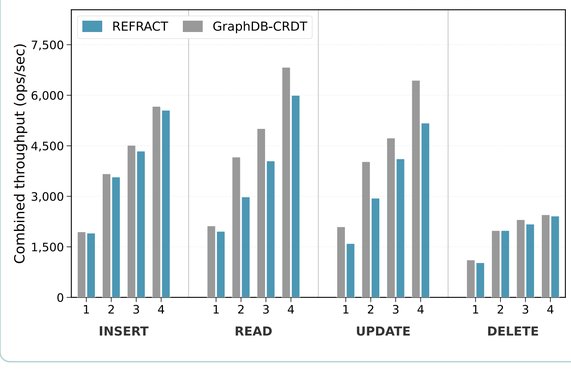


Scaling with schema size (1K nodes)

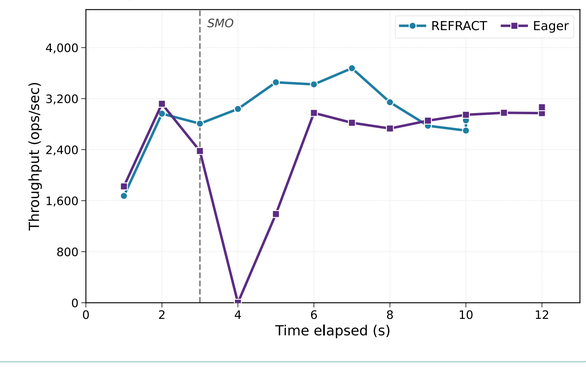


Replication and Uptime

Aggregate throughput, 1–4 replicas
(20K nodes, 12K ops per replica)



Schema evolution at 3 s
Eager blocks (20K nodes, 95 % reads)

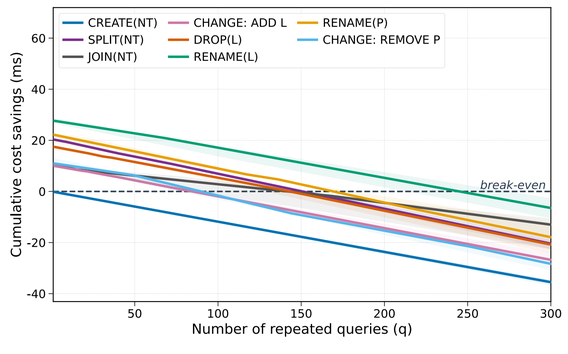


Memory and Break-Even Analysis

Replicated storage footprint
N = node count, T = schema size

		Size (MB)		Overhead	
N	T	GraphDB-CRDT	REFRACT	MB	%
100	21	0.076	0.087	0.011	14
1K	21	1.4	1.4	0.011	0.8
2.5K	21	5.4	5.4	0.011	0.2
10K	21	40.1	40.1	0.011	0.03
1K	100	1.4	1.5	0.1	7
1K	1K	1.4	2.0	0.6	43
1K	2.5K	1.4	2.9	1.5	107
1K	10K	1.4	7.6	6.2	443

Break-even #queries
Lazy pays off for expensive SMOs (1K nodes)



Code & Data

github.com/MichaLandgram/REFRACT

Paper

doi.org/10.1145/3799682.3840015

References

[1] A. Bonifati et al. "Schema Validation and Evolution for Graph Databases". *ER*. 2019. [2] B. F. Cooper et al. "Benchmarking Cloud Serving Systems with YCSB". *Proceedings of the 1st ACM Symposium on Cloud Computing*. 2010. [3] O. Erling et al. "The LDBC Social Network Benchmark: Interactive Workload". *SIGMOD Conference*. 2015. [4] K. Jahns et al. *Yjs | Homepage*. [5] G. Litt et al. "Cambria: Schema Evolution in Distributed Systems with Edit Lenses". *PaPoC@EuroSys*. 2021. [6] Neo4j. *Set Graph Types - Cypher Manual*. [7] M. Shapiro et al. "Conflict-Free Replicated Data Types". *SSS*. 2011.